

Experiment No. 2

To implement and analyze text vectorization techniques including Bag-of-Words (count occurrence and normalized count), TF-IDF (Term Frequency-Inverse Document Frequency), and Word2Vec embeddings for converting text data into numerical representations

Install & Import libraries

```
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from gensim.models import Word2Vec
import matplotlib.pyplot as plt
```

```
# Our sample sentences (documents)
documents = [
    "I am a big fan of Biryani",
    "Dal rice is my favorite food",
    "I love to watch movies",
    "Cricket is a famous sport",
    "I love Biryani and Cricket"
]
```

PART 1: Bag of Words (BoW)

```
# Create a CountVectorizer - our word counting machine!
count_vectorizer = CountVectorizer()
```

```
# Transform our sentences into numbers (counts)
count_matrix = count_vectorizer.fit_transform(documents)
```

```
# Get the vocabulary (all unique words)
vocabulary = count_vectorizer.get_feature_names_out()
```

```
print("Our Vocabulary (All Unique Words):")
print(vocabulary)
print(f"\n Total unique words: {len(vocabulary)}")
```

```
Our Vocabulary (All Unique Words):
['am' 'and' 'big' 'biryani' 'cricket' 'dal' 'famous' 'fan' 'favorite'
 'food' 'is' 'love' 'movies' 'my' 'of' 'rice' 'sport' 'to' 'watch']
```

```
Total unique words: 19
```

```
count_df = pd.DataFrame(
    count_matrix.toarray(), # Convert to regular array
    columns=vocabulary,     # Column names = words
    index=[f"Doc {i+1}" for i in range(len(documents))] # Row names = documents
)
```

```
print("\n BAG OF WORDS - Count Occurrence (How many times each word appears):")
print("=" * 70)
print(count_df)
print("\n Each number shows how many times that word appears in each document!")
```

```
BAG OF WORDS - Count Occurrence (How many times each word appears):
=====
      am  and  big  biryani  cricket  dal  famous  fan  favorite  food  is  \
Doc 1   1   0   1         1         0   0         0   1         0   0   0
Doc 2   0   0   0         0         0   1         0   0         1   1   1
Doc 3   0   0   0         0         0   0         0   0         0   0   0
Doc 4   0   0   0         0         1   0         1   0         0   0   1
Doc 5   0   1   0         1         1   0         0   0         0   0   0

      love  movies  my  of  rice  sport  to  watch
Doc 1     0       0   0   1   0     0   0     0
Doc 2     0       0   0   1   0     0   0     0
Doc 3     1       1   0   0   0     0   1     1
Doc 4     0       0   0   0   0     1   0     0
Doc 5     1       0   0   0   0     0   0     0
```

```
Each number shows how many times that word appears in each document!
```

```
# Method 1: L1 Normalization (divide by sum of all words)
# This gives us the PROPORTION of each word
```

```
# Get the raw counts
raw_counts = count_matrix.toarray()

# Calculate sum of each row (total words in each document)
row_sums = raw_counts.sum(axis=1, keepdims=True)

# Divide each count by the total (L1 Normalization)
l1_normalized = raw_counts / row_sums

# Create a nice table
l1_norm_df = pd.DataFrame(
    l1_normalized,
    columns=vocabulary,
    index=[f"Doc {i+1}" for i in range(len(documents))]
)

print("NORMALIZED COUNT (L1 Normalization - Proportions):")
print("=" * 70)
print(l1_norm_df.round(3)) # Round to 3 decimal places for clarity
print("\n Now each row adds up to 1.0 (100%)!")
print(f"Row sums: {l1_norm_df.sum(axis=1).values}")
```

```
NORMALIZED COUNT (L1 Normalization - Proportions):
=====
      am  and  big  biryani  cricket  dal  famous  fan  favorite  food \
Doc 1  0.2  0.00  0.2   0.20   0.00  0.000   0.00  0.2   0.000  0.000
Doc 2  0.0  0.00  0.0   0.00   0.00  0.167   0.00  0.0   0.167  0.167
Doc 3  0.0  0.00  0.0   0.00   0.00  0.000   0.00  0.0   0.000  0.000
Doc 4  0.0  0.00  0.0   0.00   0.25  0.000   0.25  0.0   0.000  0.000
Doc 5  0.0  0.25  0.0   0.25   0.25  0.000   0.00  0.0   0.000  0.000

      is  love  movies   my  of  rice  sport  to  watch
Doc 1  0.000  0.00   0.00  0.000  0.2  0.000  0.00  0.00  0.00
Doc 2  0.167  0.00   0.00  0.167  0.0  0.167  0.00  0.00  0.00
Doc 3  0.000  0.25   0.25  0.000  0.0  0.000  0.00  0.25  0.25
Doc 4  0.250  0.00   0.00  0.000  0.0  0.000  0.25  0.00  0.00
Doc 5  0.000  0.25   0.00  0.000  0.0  0.000  0.00  0.00  0.00

Now each row adds up to 1.0 (100%)!
Row sums: [1. 1. 1. 1. 1.]
```

```
# Method 2: L2 Normalization (using Euclidean/geometric normalization)
# This is like making each document vector have length = 1
```

```
from sklearn.preprocessing import normalize
```

```
# L2 Normalize the count matrix
l2_normalized = normalize(count_matrix, norm='l2')
```

```
# Create a nice table
l2_norm_df = pd.DataFrame(
    l2_normalized.toarray(),
    columns=vocabulary,
    index=[f"Doc {i+1}" for i in range(len(documents))]
)
```

```
print("NORMALIZED COUNT (L2 Normalization - Unit Vectors):")
print("=" * 70)
print(l2_norm_df.round(3))
print("\n L2 normalization is commonly used in machine learning!")
```

```
NORMALIZED COUNT (L2 Normalization - Unit Vectors):
=====
      am  and  big  biryani  cricket  dal  famous  fan  favorite  \
Doc 1  0.447  0.0  0.447  0.447   0.0  0.000   0.0  0.447  0.000
Doc 2  0.000  0.0  0.000  0.000   0.0  0.408   0.0  0.000  0.408
Doc 3  0.000  0.0  0.000  0.000   0.0  0.000   0.0  0.000  0.000
Doc 4  0.000  0.0  0.000  0.000   0.5  0.000   0.5  0.000  0.000
Doc 5  0.000  0.5  0.000  0.500   0.5  0.000   0.0  0.000  0.000

      food  is  love  movies   my  of  rice  sport  to  watch
Doc 1  0.000  0.000  0.0   0.0  0.000  0.447  0.000  0.0  0.0  0.0
Doc 2  0.408  0.408  0.0   0.0  0.408  0.000  0.408  0.0  0.0  0.0
Doc 3  0.000  0.000  0.5   0.5  0.000  0.000  0.000  0.0  0.5  0.5
Doc 4  0.000  0.500  0.0   0.0  0.000  0.000  0.000  0.5  0.0  0.0
Doc 5  0.000  0.000  0.5   0.0  0.000  0.000  0.000  0.0  0.0  0.0
```

```
L2 normalization is commonly used in machine learning!
```

```
# Create a TF-IDF Vectorizer - our "importance finder" machine!
```

```
tfidf_vectorizer = TfidfVectorizer()
```

```
# Transform our sentences
```

```
tfidf_matrix = tfidf_vectorizer.fit_transform(documents)
```

```
# Get vocabulary
tfidf_vocabulary = tfidf_vectorizer.get_feature_names_out()

# Create a nice table
tfidf_df = pd.DataFrame(
    tfidf_matrix.toarray(),
    columns=tfidf_vocabulary,
    index=[f"Doc {i+1}" for i in range(len(documents))]
)

print(" TF-IDF SCORES (How Important Each Word Is):")
print("=" * 70)
print(tfidf_df.round(3))
```

```
TF-IDF SCORES (How Important Each Word Is):
=====
```

	am	and	big	biryani	cricket	dal	famous	fan	favorite	\
Doc 1	0.464	0.000	0.464	0.374	0.000	0.000	0.00	0.464	0.000	
Doc 2	0.000	0.000	0.000	0.000	0.000	0.421	0.00	0.000	0.421	
Doc 3	0.000	0.000	0.000	0.000	0.000	0.000	0.00	0.000	0.000	
Doc 4	0.000	0.000	0.000	0.000	0.444	0.000	0.55	0.000	0.000	
Doc 5	0.000	0.582	0.000	0.470	0.470	0.000	0.00	0.000	0.000	

	food	is	love	movies	my	of	rice	sport	to	watch
Doc 1	0.000	0.000	0.000	0.000	0.000	0.464	0.000	0.00	0.000	0.000
Doc 2	0.421	0.339	0.000	0.000	0.421	0.000	0.421	0.00	0.000	0.000
Doc 3	0.000	0.000	0.422	0.523	0.000	0.000	0.000	0.00	0.523	0.523
Doc 4	0.000	0.444	0.000	0.000	0.000	0.000	0.000	0.55	0.000	0.000
Doc 5	0.000	0.000	0.470	0.000	0.000	0.000	0.000	0.00	0.000	0.000

```
# Let's understand the IDF values (how rare each word is)
idf_values = tfidf_vectorizer.idf_

idf_df = pd.DataFrame({
    'Word': tfidf_vocabulary,
    'IDF Score': idf_values
}).sort_values('IDF Score', ascending=True)

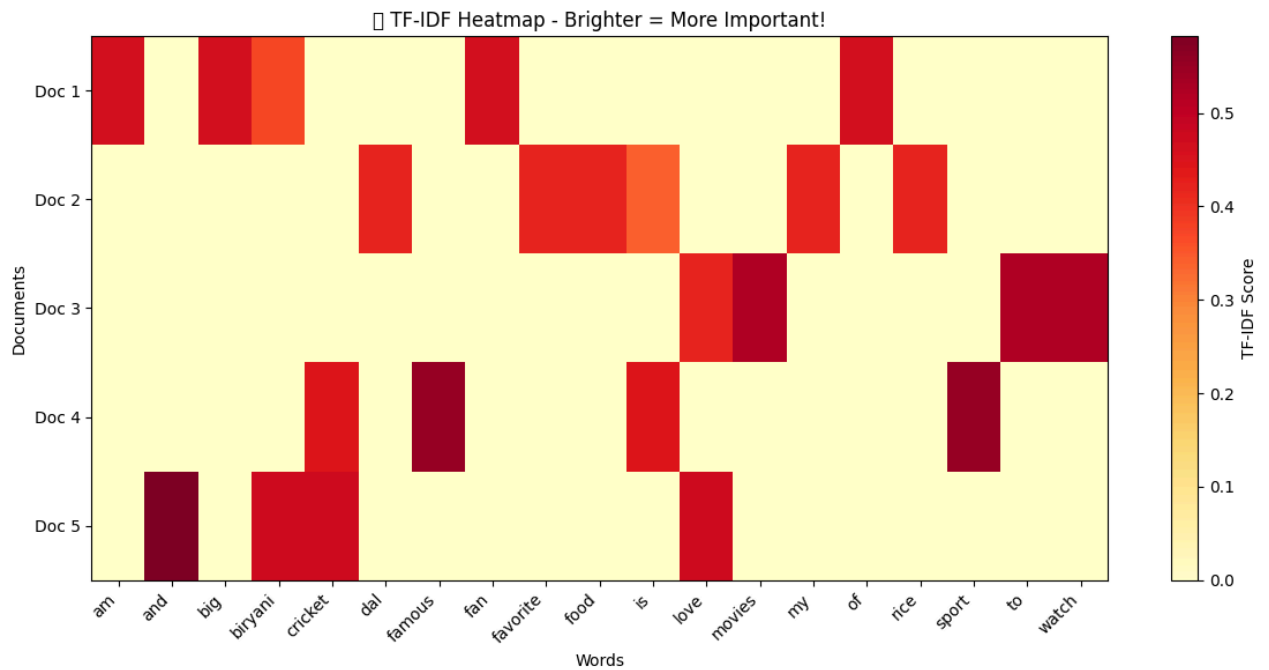
print("\n IDF Scores (Lower = More Common, Higher = More Rare):")
print("=" * 50)
print(idf_df.round(3).to_string(index=False))
```

```
IDF Scores (Lower = More Common, Higher = More Rare):
=====
```

Word	IDF Score
biryani	1.693
cricket	1.693
love	1.693
is	1.693
am	2.099
sport	2.099
rice	2.099
of	2.099
my	2.099
movies	2.099
food	2.099
favorite	2.099
fan	2.099
famous	2.099
dal	2.099
big	2.099
and	2.099
to	2.099
watch	2.099

```
# Visualize TF-IDF scores with a heatmap
plt.figure(figsize=(12, 6))
plt.imshow(tfidf_df.values, cmap='YlOrRd', aspect='auto')
plt.colorbar(label='TF-IDF Score')
plt.xticks(range(len(tfidf_vocabulary)), tfidf_vocabulary, rotation=45, ha='right')
plt.yticks(range(len(documents)), [f"Doc {i+1}" for i in range(len(documents))])
plt.xlabel('Words')
plt.ylabel('Documents')
plt.title('🌟 TF-IDF Heatmap - Brighter = More Important!')
plt.tight_layout()
plt.show()
```

```
C:\Users\Siddharth Raut\AppData\Local\Temp\ipykernel_15520\138585712.py:10: UserWarning: Glyph 127775 (\N{GLOWING STAR}) mis
plt.tight_layout()
C:\Users\Siddharth Raut\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:152: UserWarning: Glyph 12
fig.canvas.print_figure(bytes_io, **kw)
```



#PART 3: Word2Vec

```
# First, we need to prepare our text
# Word2Vec needs a list of lists (each sentence as a list of words)

# Tokenize (split into words) and convert to lowercase
tokenized_docs = [doc.lower().split() for doc in documents]

print(" Tokenized Documents (Words as Lists):")
print("=" * 50)
for i, tokens in enumerate(tokenized_docs, 1):
    print(f"Doc {i}: {tokens}")
```

```
Tokenized Documents (Words as Lists):
=====
Doc 1: ['i', 'am', 'a', 'big', 'fan', 'of', 'biryani']
Doc 2: ['dal', 'rice', 'is', 'my', 'favorite', 'food']
Doc 3: ['i', 'love', 'to', 'watch', 'movies']
Doc 4: ['cricket', 'is', 'a', 'famous', 'sport']
Doc 5: ['i', 'love', 'biryani', 'and', 'cricket']
```

```
# Train Word2Vec model
# Think of this as "teaching" the computer about word relationships!

word2vec_model = Word2Vec(
    sentences=tokenized_docs, # Our training data
    vector_size=50,           # Each word becomes 50 numbers (dimensions)
    window=3,                 # Look at 3 words before and after
    min_count=1,               # Include words that appear at least 1 time
    workers=4,                 # Use 4 CPU cores for speed
    epochs=100                 # Train 100 times over the data
)
```

```
# Let's see the words in our vocabulary
words_in_vocab = list(word2vec_model.wv.key_to_index.keys())
print(" Words in Word2Vec Vocabulary:")
print(words_in_vocab)
```

```
Words in Word2Vec Vocabulary:
['i', 'cricket', 'love', 'is', 'biryani', 'a', 'and', 'sport', 'famous', 'movies', 'watch', 'to', 'food', 'favorite', 'my',
```

```
# Get the vector (embedding) for a word
# This is the "number representation" of the word!
```

```
word = "biryani"
word_vector = word2vec_model.wv[word]
```

```
print(f" Vector for '{word}':")
print(f"   Shape: {word_vector.shape} (50 numbers!)")
print(f"   First 10 values: {word_vector[:10].round(3)}")
print("\n Each word is now represented by 50 numbers!")
```

```
Vector for 'biryani':
  Shape: (50,) (50 numbers!)
  First 10 values: [-0.      0.006 -0.014 -0.003  0.015  0.014 -0.007  0.005 -0.017  0.012]
```

Each word is now represented by 50 numbers!

```
# Create a DataFrame showing all word embeddings
embedding_dict = {}
for word in words_in_vocab:
    embedding_dict[word] = word2vec_model.wv[word]

# Show first 10 dimensions for each word
embedding_df = pd.DataFrame(embedding_dict).T
embedding_df.columns = [f"Dim_{i+1}" for i in range(embedding_df.shape[1])]

print("🔴 Word Embeddings (First 10 Dimensions):")
print("=" * 70)
print(embedding_df.iloc[:, :10].round(3))
```

```
🔴 Word Embeddings (First 10 Dimensions):
=====
      Dim_1 Dim_2 Dim_3 Dim_4 Dim_5 Dim_6 Dim_7 Dim_8 Dim_9 \
i      -0.001 0.001 0.010 0.018 -0.019 -0.014 0.013 0.018 -0.010
cricket -0.016 0.009 -0.008 0.002 0.017 -0.009 0.009 -0.014 -0.007
love    -0.017 0.007 0.010 0.012 0.015 -0.012 0.002 0.012 -0.006
is       0.016 -0.019 -0.000 0.007 -0.002 0.017 0.018 0.013 -0.002
biryani -0.000 0.006 -0.014 -0.003 0.015 0.014 -0.007 0.005 -0.017
a        0.003 -0.005 -0.014 -0.016 -0.018 -0.012 -0.004 -0.009 -0.013
and      -0.017 0.019 -0.000 -0.004 0.009 -0.008 0.006 0.014 0.012
sport    0.009 0.000 -0.019 -0.019 -0.012 -0.000 0.004 0.019 0.011
famous   -0.014 0.002 -0.014 -0.004 0.007 0.012 0.002 0.004 -0.008
movies   0.008 0.009 0.020 -0.009 -0.003 -0.015 -0.019 -0.018 -0.002
watch    -0.017 0.004 -0.002 -0.019 -0.019 -0.003 0.009 0.007 -0.013
to        0.005 -0.017 0.016 0.017 -0.019 0.005 0.020 -0.015 -0.014
food      0.016 -0.009 -0.002 0.002 -0.000 0.002 0.012 0.000 -0.007
favorite -0.001 -0.018 -0.017 0.006 -0.016 -0.018 -0.005 -0.017 -0.014
my        0.016 -0.009 0.018 0.017 -0.009 0.001 0.009 -0.008 -0.011
rice     -0.018 -0.003 0.019 -0.015 -0.011 0.018 -0.018 0.008 0.001
dal      -0.019 0.018 0.008 0.019 0.013 0.006 0.020 -0.009 -0.014
of       -0.000 0.001 -0.000 -0.009 0.009 -0.005 0.004 0.001 0.012
fan      -0.010 -0.013 -0.016 0.017 -0.004 -0.014 -0.008 0.010 -0.006
big       0.005 0.016 -0.003 0.016 0.007 -0.016 -0.008 -0.005 0.010
am       0.014 -0.003 0.016 -0.019 -0.016 -0.013 -0.008 0.010 -0.008

      Dim_10
i      -0.008
cricket 0.019
love    -0.012
is       0.015
biryani 0.012
a       -0.008
and     -0.015
sport   -0.009
famous  0.014
movies  -0.013
watch   -0.014
to      -0.016
food    -0.003
favorite -0.017
my      -0.013
rice    0.013
dal     0.008
of      -0.014
fan     -0.008
big     -0.002
am      -0.017
```

```
# Find similar words!
# This is the magic of Word2Vec - it knows which words are "friends"!

print("🔵 Finding Similar Words:")
print("=" * 50)

test_words = ['biryani', 'movies', 'cricket']

for word in test_words:
    try:
```

```

similar = word2vec_model.wv.most_similar(word, topn=3)
print(f"\n Words similar to '{word}':")
for similar_word, similarity in similar:
    print(f"    - {similar_word}: {similarity:.3f} (similarity score)")
except KeyError:
    print(f"    Word '{word}' not in vocabulary")

print("\n Higher similarity score = more related words!")

```

🔍 Finding Similar Words:
=====

```

Words similar to 'biryani':
- love: 0.172 (similarity score)
- favorite: 0.164 (similarity score)
- sport: 0.141 (similarity score)

```

```

Words similar to 'movies':
- a: 0.225 (similarity score)
- favorite: 0.128 (similarity score)
- my: 0.120 (similarity score)

```

```

Words similar to 'cricket':
- dal: 0.235 (similarity score)
- big: 0.166 (similarity score)
- fan: 0.157 (similarity score)

```

Higher similarity score = more related words!

```

# Visualize word embeddings in 2D
# We'll reduce 50 dimensions to 2 dimensions so we can see them!

from sklearn.decomposition import PCA

# Get all word vectors
word_vectors = np.array([word2vec_model.wv[word] for word in words_in_vocab])

# Reduce to 2D using PCA
pca = PCA(n_components=2)
word_vectors_2d = pca.fit_transform(word_vectors)

# Plot
plt.figure(figsize=(10, 8))
plt.scatter(word_vectors_2d[:, 0], word_vectors_2d[:, 1], c='blue', s=100)

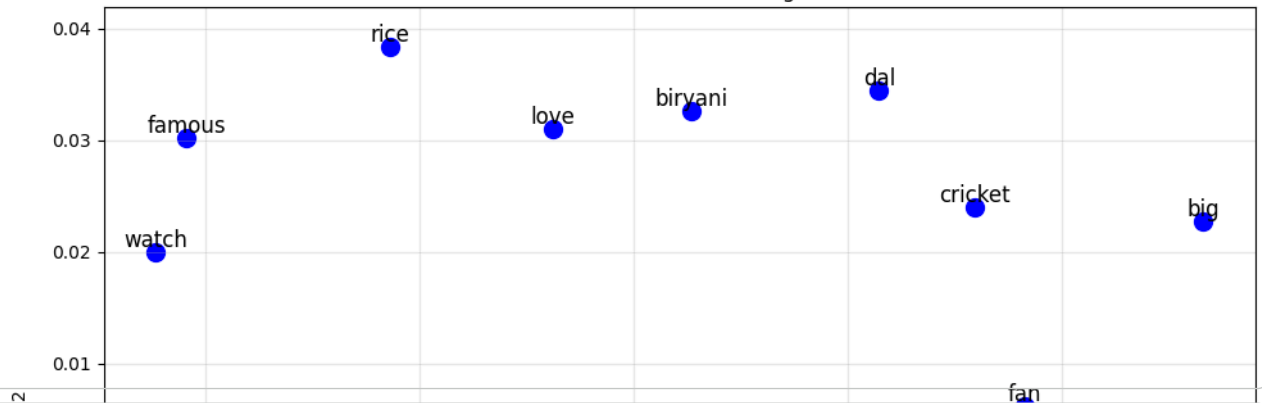
# Add word labels
for i, word in enumerate(words_in_vocab):
    plt.annotate(word, (word_vectors_2d[i, 0], word_vectors_2d[i, 1]),
                  fontsize=12, ha='center', va='bottom')

plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.title(' Word2Vec: Words in 2D Space\n(Similar words are closer together!)')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(" Words that appear in similar contexts are placed close together!")

```

Word2Vec: Words in 2D Space
(Similar words are closer together!)



```
# Create document embeddings from word embeddings
# We can represent a whole sentence by AVERAGING the word vectors!

def get_document_embedding(doc_tokens, model):
    """Average word vectors to get document vector"""
    word_vectors = []
    for word in doc_tokens:
        if word in model.wv:
            word_vectors.append(model.wv[word])
    if word_vectors:
        return np.mean(word_vectors, axis=0)
    else:
        return np.zeros(model.wv.vector_size)

# Get embeddings for all documents
doc_embeddings = [get_document_embedding(doc, word2vec_model) for doc in tokenized_docs]

# Create DataFrame
doc_embedding_df = pd.DataFrame(
    doc_embeddings,
    index=[f"Doc {i+1}" for i in range(len(documents))]
)
doc_embedding_df.columns = [f"Dim_{i+1}" for i in range(doc_embedding_df.shape[1])]

print(" Document Embeddings (First 10 Dimensions):")
print("=" * 70)
print(doc_embedding_df.iloc[:, :10].round(3))
print("\n Each document is now represented by 50 numbers (we show first 10)!")
```

```
Document Embeddings (First 10 Dimensions):
=====
   Dim_1 Dim_2 Dim_3 Dim_4 Dim_5 Dim_6 Dim_7 Dim_8 Dim_9 Dim_10
Doc 1  0.001  0.000 -0.003  0.001 -0.004 -0.009 -0.003  0.005 -0.005 -0.006
Doc 2  0.002 -0.007  0.004  0.006 -0.004  0.004  0.006 -0.002 -0.008  0.001
Doc 3 -0.005  0.001  0.011  0.004 -0.009 -0.008  0.005  0.001 -0.009 -0.013
Doc 4 -0.001 -0.003 -0.011 -0.006 -0.002  0.001  0.006  0.003 -0.004  0.006
Doc 5 -0.010  0.008 -0.000  0.005  0.008 -0.006  0.005  0.007 -0.006 -0.001
```

Each document is now represented by 50 numbers (we show first 10)!

Start coding or [generate](#) with AI.